RIPSIM TECHNOLOGIES · ENGINEERING

Epic Proposal: Project Sift — Binary Payload Key Extraction Pipeline

STATUS	**Draft**	PRIORITY	**High**
TARGET QUARTER	**Next quarter**	REQUESTED BY	**Data Platform / Integrations**
STACK	**Backend in your language of choice · Frontend in React**		

Background

Several of our upstream partners deliver data as opaque binary payloads. Somewhere inside these payloads is JSON — sometimes one document, sometimes several, sometimes fragments. Downstream teams keep asking us for "the keys" from that JSON so they can build schema catalogs, detect drift, and route records.

Today this is done with ad-hoc scripts that nobody trusts. We want a real pipeline.

Goal

Build an ETL pipeline that:

1. **Extracts** — ingests raw binary input from partner feeds.
2. **Transforms** — locates JSON content within the binary and extracts the keys present in that JSON.
3. **Loads** — makes the extracted key information available to downstream consumers.

Plus a **React dashboard** where an operator can upload payloads and watch batches complete.

The backend may be written in any language you are productive in, and should be designed so other teams can integrate with it.

In Scope

- Ingestion of binary payloads (sizes vary; some are "large").
- Detection and extraction of JSON embedded in binary content.
- Extraction of keys from the discovered JSON.

©2026 RiPSIM Technologies Inc.

- An API surface (library, service, or both — proposal expected) for downstream consumption.
- A React dashboard: upload payloads, see each batch's status (busy / completed), and browse the extracted keys once done.
- Basic observability: we should know when extraction fails and why.

Out of Scope (probably)

- Extracting JSON *values* (for now — but don't paint us into a corner).
- Decrypting or decompressing payloads (TBD — some partners may compress).
- Real-time guarantees.

Success Criteria

- Downstream teams can get key data without reading our source code.
- The pipeline handles malformed partner data without falling over.
- Reasonable performance on large payloads.

Proposed API (v0 sketch)

A previous engineer left this sketch behind. Treat it as a starting point, not a spec.

Upload a batch of payloads:

```
POST /v1/batches
Content-Type: application/octet-stream

202 Accepted
{ "batchId": "b_7f3a92" }
```

Poll for results:

```
GET /v1/batches/{batchId}

200 OK
{
  "batchId": "b_7f3a92",
  "status": "busy" | "completed",
  "extracts": [
```

 ©2026 RiPSIM Technologies Inc.

```
    { "payloadIndex": 0, "keys": ["orderId"] },
    { "payloadIndex": 1, "keys": ["sku", "qty", "warehouse", "zone"] }
  ]
}
```

The previous engineer also sketched a service interface (in C#, but the contract is language-agnostic — translate it to your stack):

```
public interface IExtractionService
{
    Task<string> UploadBatchAsync(Stream payloads, CancellationToken ct = default);
    Task<BatchResult> GetResultsAsync(string batchId, CancellationToken ct = default);
}

public sealed record BatchResult(string BatchId, BatchStatus Status,
IReadOnlyList<PayloadExtract> Extracts);
public sealed record PayloadExtract(int PayloadIndex, IReadOnlyList<string> Keys);
public enum BatchStatus { Busy, Completed }
```

Sample Payloads

Three real partner payloads for development and testing, as hex. Each contains binary framing around embedded content.

Payload A

```
DE AD BE EF 01 00 00 12 9C 41 F3 07 7B 22 6F 72
64 65 72 49 64 22 3A 22 41 31 42 32 2D 39 46 22
7D 00 00 FF FF C8 33
```

For reference, your solution should extract exactly one key from this payload: `orderId`.

Payload B

```
CA FE BA BE 02 00 00 3B 11 E0 7B 22 73 6B 75 22
3A 22 58 2D 31 30 30 22 2C 22 71 74 79 22 3A 35
2C 22 77 61 72 65 68 6F 75 73 65 22 3A 7B 22 7A
6F 6E 65 22 3A 22 43 34 22 7D 7D 0D 0A 00 00 7A
7A
```

 ©2026 RiPSIM Technologies Inc.

This payload contains multiple keys. Report what your solution extracts, and be prepared to justify your definition of a "key."

Payload C

```
17 03 03 00 7B 42 42 00 E1 7B 22 6E 6F 74 65 22
3A 22 75 73 65 20 7B 20 77 69 74 68 20 63 61 72
65 22 2C 22 73 74 61 74 75 73 22 3A 22 6F 6B 22
7D 00 04 D2 00
```

Also contains multiple keys. Partner data is representative of production — treat it accordingly.

Open Questions

- What exactly counts as a "key"? Top-level only? Nested? Arrays of objects?
- How do partners deliver payloads — files, streams, message bus?
- How do multiple payloads get delimited inside one upload body?
- Is the v0 sketch good enough to build on, or does it need rework first?
- What should the dashboard show while a batch is busy?

Deliverables

- Technical design covering the API contract and pipeline architecture.
- Working implementation of the core extraction/transform stage, in your language of choice.
- A React dashboard for uploading payloads and viewing batch status and extracted keys.
- Tests demonstrating behavior on well-formed, malformed, and adversarial inputs.

©2026 RiPSIM Technologies Inc.